Министерство науки и высшего образования РФ
АРЗАМАССКИЙ ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ (ФИЛИАЛ)
ФЕДЕРАЛЬНОГО ГОСУДАРСТВЕННОГО БЮДЖЕТНОГО
ОБРАЗОВАТЕЛЬНОГО УЧРЕЖДЕНИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «НИЖЕГОРОДСКИЙ
ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ ИМ. Р.Е.АЛЕКСЕЕВА»
(АПИ НГТУ)

Лабораторная работа по дисциплине «Логика и архитектура вычислительных средств»

Выполнила:
студентка гр. МА26ПМ
Фролова Д.А.

Проверил:
Комаров А.О.

Арзамас, 2026

RL-агент для Pong (DQN) – архитектурный проект

Проект: обучение RL-агента игре в Pong с production-подобной архитектурой: тренер, inference-сервис, метрики/эксперимент-трекинг, обмен событиями через Kafka.

1. Цель и проблема проекта

Цель – построить полный жизненный цикл RL-системы: обучить агента играть в Pong и довести модель до эксплуатации через API, с трекингом экспериментов и метрик.

Проблема. Обучение RL в ноутбуке легко, а эксплуатация – нет: нет истории экспериментов, метрики хранятся «где-то», предикт не отделён от обучения, сервисы не масштабируются. Проект решает это: обучение отделено от инференса, все события тренировки идут через Kafka в аналитическое хранилище, а каждый эксперимент фиксируется в MLflow.

2. Требования и ограничения

Функциональные требования:

1. F1. Игровая среда Pong в Gymnasium API (+ поддержка Breakout через ALE).
2. F2. Обучение DQN (основной) и PPO (альтернатива) через Stable-Baselines3.
3. F3. Inference-сервис: по наблюдению (4×84×84) вернуть действие.
4. F4. Метрики эпизодов публикуются в Kafka и складываются в PostgreSQL.
5. F5. Трекинг экспериментов и артефактов в MLflow (UI на порту 5001).
6. F6. Smoke-тест проверяет связность сервисов реальными HTTP-запросами.

7. F7. Юнит-тесты логики окружения, паттернов и схем сообщений.

Нефункциональные требования:

– CPU-only обучение (воспроизводимость, без GPU-требований).

– Отказоустойчивость: недоступность Kafka/MLflow не роняет тренировку (best-effort телеметрия).

– Идемпотентный запуск docker compose up с healthcheck'ами и ожиданием зависимостей.

Ограничения:

– Датасет генерируется средой (self-play симуляция), внешний датасет не нужен.

– Обучение DQN 200k шагов на CPU занимает ориентировочно 40–90 минут; для проверки pipeline используется быстрый прогон MAX_EPISODES / TOTAL_TIMESTEPS.

– Противник – простая эвристика с шумом (не обучаемый самоплей).

3. Входные и выходные данные

Входные данные:

– Наблюдение агента: uint8[4, 84, 84] – стек 4 последних кадров в градациях серого (frame stacking даёт скорость мяча).

– Действие: int из {0: stay, 1: up, 2: down}.

– HTTP-запрос inference: POST /predict {"observation": [[[...]]]}.

Выходные данные:

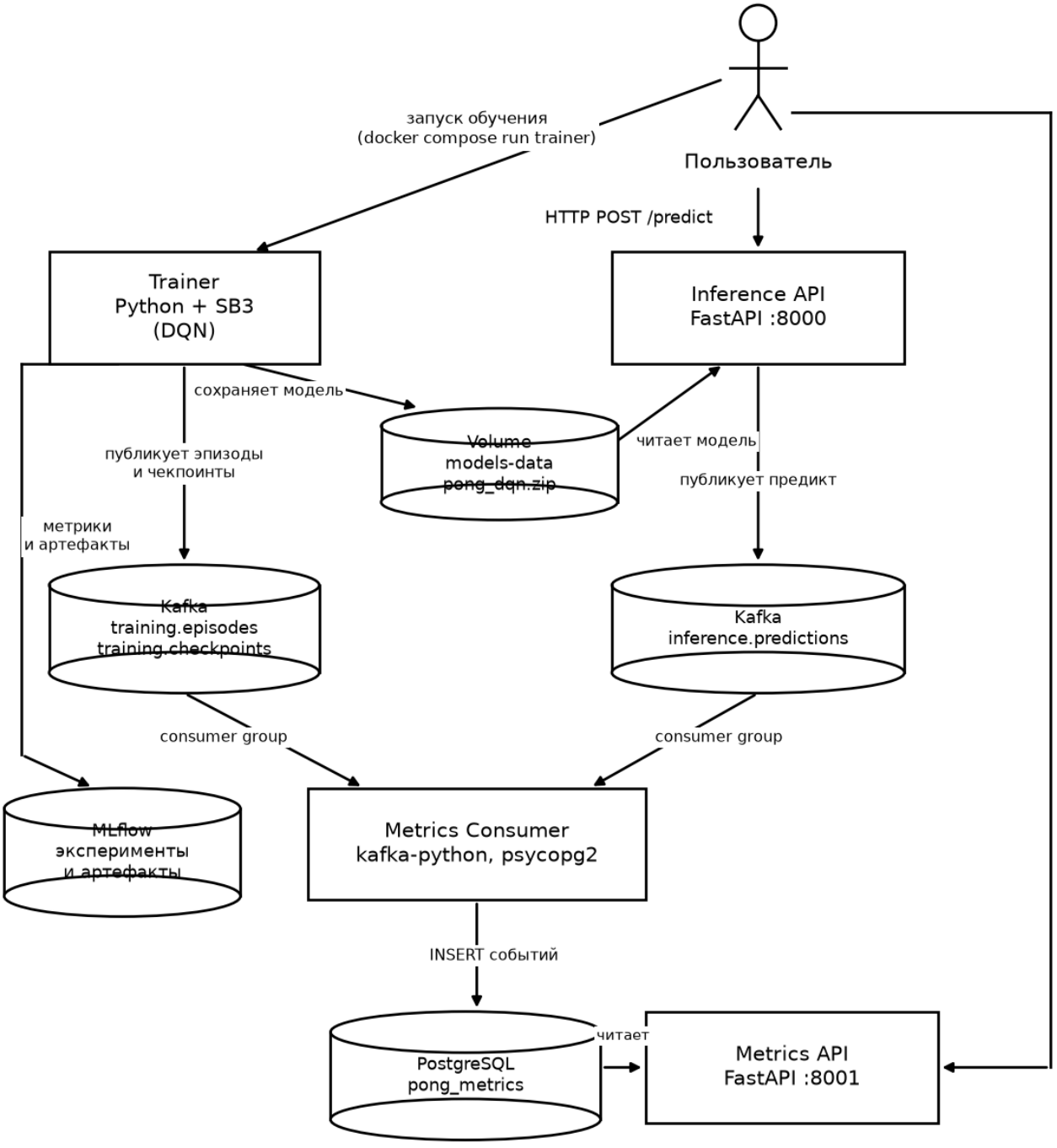
– Действие агента и латентность: {"request_id", "action", "action_name", "latency_ms", "model"}.

– События Kafka: training.episodes, training.checkpoints, inference.predictions, training.commands.

– Записи PostgreSQL: episodes, checkpoints, inference_logs.

– Артефакты MLflow: метрики эпизодов, параметры, файл модели (DQN zip).

4. Архитектурная схема (C4)



5. Микросервисы и границы ответственности

Сервис	Технология	Ответственность	Граница
trainer	Python, SB3	обучение, чекпоинты, публикация событий	не отвечает на HTTP-запросы
inference	FastAPI	только предикт обученной модели	не обучает, не пишет в БД
metrics-consumer	kafka-python, psycopg2	только транспорт событий в БД	не агрегирует
metrics-api	FastAPI	только чтение/агрегация из БД	не пишет
kafka/zookeeper	Confluent images	шина событий	–
postgres	Postgres 15	аналитическое хранилище	–
mlflow	MLflow server	трекинг экспериментов	–

Такое разделение позволяет масштабировать инференс и тренер независимо и заменять любой сервис без правки остальных (только контракт топиков/API).

6. Kafka: топики, продюсеры, консьюмеры

Топик	Producer	Consumer (group)	Формат (JSON)
training.episodes	trainer	metrics-consumer (metrics-consumer-group)	{"experiment_id","run_id","episode", "steps","reward","epsilon","loss","ts"}
training.checkpoints	trainer	metrics-consumer (metrics-consumer-group)	{"experiment_id","run_id","step", "artifact_uri","ts"}
inference.predictions	inference	metrics-consumer (metrics-consumer-group)	{"request_id","model_name","action", "action_name","latency_ms","ts"}
training.commands	оператор/ клиент	trainer-daemon (trainer-command-group)	{"command":"train|evaluate","params":{ ...},"request_id","ts"}

Примеры сообщений:

```
{"experiment_id": "dqn-PongPygame-v0", "run_id": "c24bbcc8",
 "episode": 42, "steps": 2000, "reward": 11.91, "epsilon": 0.05, "loss": 0.0,
"ts": <unix-время эпизода 42>}
```

7. Схема БД и индексы

```
CREATE TABLE IF NOT EXISTS episodes (
    id            BIGSERIAL PRIMARY KEY,
    experiment_id TEXT        NOT NULL,
    run_id        TEXT,
    episode       INTEGER     NOT NULL,
    steps         INTEGER     NOT NULL,
    reward        REAL        NOT NULL,
    epsilon       REAL,
    loss          REAL,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_episodes_exp_episode ON episodes (experiment_id,
episode DESC);
CREATE INDEX IF NOT EXISTS idx_episodes_created     ON episodes (created_at);

CREATE TABLE IF NOT EXISTS checkpoints (
    id BIGSERIAL PRIMARY KEY, experiment_id TEXT NOT NULL, run_id TEXT,
    step INTEGER NOT NULL, artifact_uri TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS inference_logs (
    id BIGSERIAL PRIMARY KEY, request_id TEXT NOT NULL, model_name TEXT,
    action INTEGER NOT NULL, action_name TEXT, latency_ms REAL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
CREATE INDEX IF NOT EXISTS idx_infer_created ON inference_logs (created_at);
CREATE INDEX IF NOT EXISTS idx_infer_model   ON inference_logs (model_name);

CREATE TABLE IF NOT EXISTS experiments (experiment_id TEXT PRIMARY KEY,
    algorithm TEXT, created_at TIMESTAMPTZ DEFAULT NOW());
```

Индексы: (experiment_id, episode DESC) – под запросы /episodes и временные ряды; по created_at – для дашборда «последние события»; по model_name – для статистики латентности по моделям. Улучшение на развитие: составной индекс (run_id, episode) для фильтрации по конкретному прогону – нужен при росте числа исторических запусков (в текущих данных 5 прогонов, см. раздел 14). Фактическая схема сохранена в artifacts/db_schema_*.txt: таблицы episodes, checkpoints, experiments, inference_logs.

8. API с примерами

Inference (порт 8000):

```
curl -s localhost:8000/health
{"status":"ok","model_loaded":true,"model":"DQN"}
```

```
curl -s -X POST localhost:8000/predict -H "Content-Type: application/json" \
  -d '{"observation": [[<84 значений>], [<84>], [<84>], [<84>]]}'
{"request_id":"cfe2af4e49a1","action":0,"action_name":"stay",
 "latency_ms":59.808,"model":"DQN"}

curl -s localhost:8000/model/info
{"algorithm":"DQN","model_path":"/models/pong_dqn.zip",
 "observation_shape":[4,84,84],"actions":{"0":"stay","1":"up","2":"down"}}
```

Ошибки: 422 – неверная форма наблюдения; 503 – модель не загружена (файл отсутствует, например до первого обучения).

Metrics (порт 8001):

```
curl -s localhost:8001/experiments
{"experiments":[{"experiment_id":"dqn-PongPygame-v0","episodes":110,
  "first_episode":1,"last_episode":52,"avg_reward":-6.504}]}

curl -s "localhost:8001/summary?experiment_id=dqn-PongPygame-v0"
{"experiment_id":"dqn-PongPygame-v0","episodes_total":110,
 "mean_reward_first_50":-9.328,"mean_reward_last_50":5.69,
 "improvement":15.019,"best_episode_reward":11.91}
```

Pong RL Inference API 1.0.0 OAS 3.1

/openapi.json

default

GET `/health` Health

Parameters | Try it out

No parameters

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value | Schema `"string"`	*No links*

GET `/model/info` Model Info

POST `/predict` Predict

Parameters | Cancel

No parameters

Request body required | application/json

```
{
  "observation": [
    [
      [
        0
      ]
    ]
  ]
}
```

Execute

Responses

Code	Description	Links
200	Successful Response Media type: application/json Controls Accept header. Example Value | Schema `"string"`	*No links*
422	Validation Error Media type: application/json Example Value | Schema `{ "detail": [ { "loc": [ "string", 0 ], "msg": "string", "type": "string" } ] }`	*No links*

Примечание по семантике агрегатов: окна first_50/last_50 считаются по всей истории эксперимента в порядке вставки записей (src/metrics/api.py: срезы rewards[:k] и rewards[-k:]). Поэтому mean_reward_first_50 отражает старт обучения (ранние прогоны), а mean_reward_last_50 – хвост последнего прогона. Улучшение: фильтр по run_id в /summary, чтобы окна считались внутри одного прогона и не зависели от количества исторических запусков.

9. Применённые паттерны

Factory Method – src/envs/factory.py, src/agents/factory.py.

– Где: создание сред (EnvFactory.create("PongPygame-v0")) и агентов (AgentFactory.create("DQN", env)).

– Зачем: переключение окружения и алгоритма конфигурацией/CLI без правки кода; новый агент добавляется декоратором @AgentFactory.register("SAC") (открытость-закрытость).

```
@classmethod
def create(cls, name, env, **overrides):
    cls._ensure_registered()
    return cls._registry[name.upper()](env=env, **overrides)
```

Strategy – src/training/strategies.py, src/envs/rewards.py.

– Где: выбор алгоритма (DQN/PPO с разными гиперпараметрами) и стратегии награды (sparse/shaped).

– Зачем: тренер и окружение не знают деталей алгоритма; сравнение стратегий награды – часть эксперимента.

```
class ShapedRewardStrategy(RewardStrategy):
    def shape(self, env, base_reward):
        diff = abs(env.ball_y - (env.left_y + env.paddle_h / 2))
        aligned = 1.0 if diff < env.paddle_h / 2 else 0.0
        return float(base_reward) + self.ALIGNMENT_BONUS * aligned
```

Observer – src/training/observers.py.

– Где: TrainerSubject рассылает события эпизода наблюдателям Console/MLflow/Kafka.

– Зачем: тренировка не зависит от способа телеметрии; телеметрия защищена try/except – сбой Kafka не роняет обучение.

```
def notify_episode(self, metrics):
    for obs in self._observers:
        try: obs.on_episode(metrics)
        except Exception as exc: log.warning("observer %s failed: %s", ...)
```

State – src/envs/game_states.py.

– Где: жизненный цикл матча Serving → Playing → GameOver.

– Зачем: переходы и правила состояний инкапсулированы в классах; окружение только делегирует self._state = self._state.handle(cmd), терминальность определяется типом состояния.

Command – src/envs/commands.py, src/training/command_loop.py.

– Где: (а) действия агента – объекты PaddleUp/PaddleDown/PaddleStay, выполняемые над env; (б) команды обучения поверх Kafka (train, evaluate).

– Зачем: действия можно логировать/реплеить, а управлять тренером – сообщением в топик без доступа к контейнеру.

10. Dockerfile, docker-compose и запуск

deploy/Dockerfile – один образ для всех Python-сервисов (torch CPU, pygame, SB3, fastapi, kafka-python, psycopg2). docker-compose.yml: 8 сервисов в сети pong-net, healthcheck-и для kafka/postgres, volumes для моделей, БД и MLflow.

```
docker compose up -d --build        # вся инфраструктура + API

# Важно: сервис trainer при up -d стартует автоматически и начинает обучение с
нуля. Чтобы не смешивать результаты прошлых прогонов в метриках, после подъёма
инфраструктуры остановите его: docker compose stop trainer. Обучение запускается
по требованию:
docker compose run --rm trainer

docker compose run --rm trainer pytest -v   # юнит-тесты (pytest)
docker compose run --rm smoke       # сквозные HTTP-запросы между контейнерами
```

Сервисы с профилем tools (tests, smoke) не стартуют автоматически при up.

11. Тестирование и результаты

– Юнит-тесты (tests/): форма наблюдения и сброс окружения, границы действий, терминальность при 11 очках, отрицательная награда за пропущенный гол, переходы State, работа Command (движение платформы), реестры Factory, roundtrip всех схем Kafka. Фактический прогон: 15 passed (artifacts/pytest.txt).

– Smoke-тест (scripts/smoke_test.py): из контейнера в сети compose делаются реальные запросы: GET /health inference, POST /predict с валидным наблюдением, GET /health и GET /experiments metrics-api. Каждый шаг печатает PASS/FAIL и тело ответа, итог – SMOKE RESULT: ALL PASS (artifacts/smoke_test.txt).

– Реальный прогон обучения: python scripts/demo_tabular.py --steps 150000 (лёгковесный MiniPong + Q-learning, без внешних зависимостей) – выполнен, метрики в docs/demo_metrics.json, график docs/training_curve.png.

– Полный DQN-прогон выполнен в docker-стеке: события тренировки прошли через Kafka в PostgreSQL (5 прогонов, 110 эпизодов), метрики доступны через metrics-api (раздел 8) и в MLflow UI (http://localhost:5001).

12. Демонстрация pipeline

Артефакты (папка artifacts/):

– docker_ps.txt – все контейнеры запущены (8 сервисов);

– network.txt – инспекция сети pong-net (подсеть 172.19.0.0/16, все сервисы в одном списке Containers);

– kafka_topics.txt – список топиков (training.episodes, training.checkpoints, inference.predictions);

– pytest.txt – отчёт юнит-тестов, 15 passed;

– smoke_test.txt – вывод smoke-теста с реальными HTTP-ответами, ALL PASS;

– db_tables.txt – список таблиц PostgreSQL;

– db_schema_*.txt – структура и индексы всех таблиц;

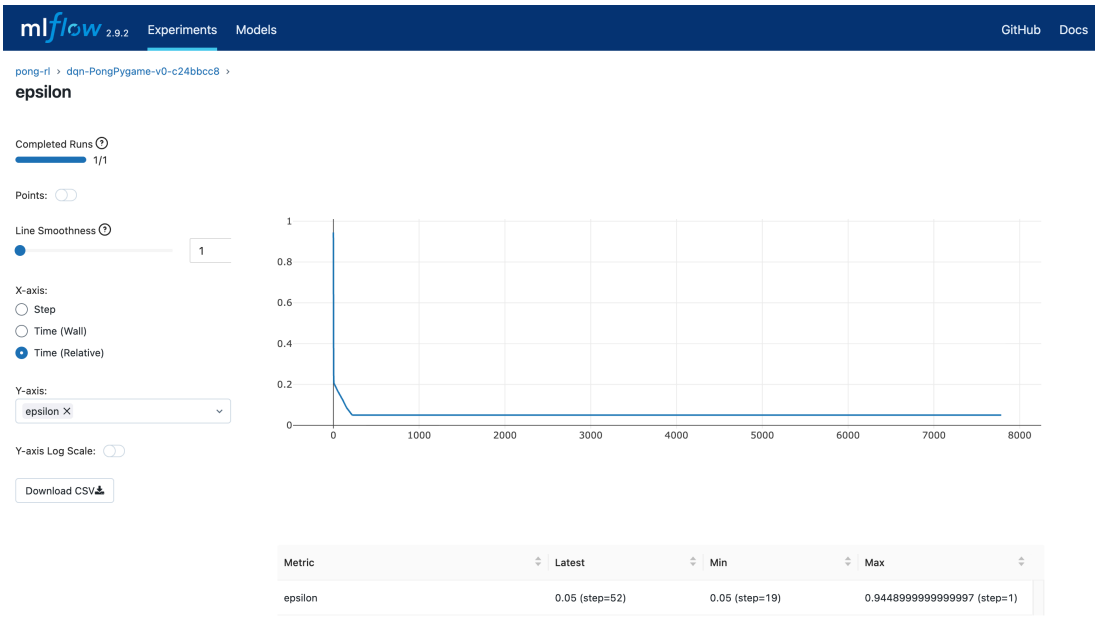
– last_run_episodes.txt – ряд наград последнего DQN-прогона (52 эпизода);

– api_summary.json – реальный ответ /summary.

```
docker compose ps
NAME                                        IMAGE
COMMAND                  SERVICE             CREATED          STATUS
PORTS
rl-pong-architecture-inference-1          rl-pong-architecture-
inference          "uvicorn src.inferen…"   inference          29
minutes ago   Up 29 minutes             0.0.0.0:8000->8000/tcp,
[::]:8000->8000/tcp
rl-pong-architecture-kafka-1              confluentinc/cp-
kafka:7.6.1             "/etc/confluent/dock…"   kafka
29 minutes ago   Up 29 minutes (healthy)   9092/tcp
rl-pong-architecture-metrics-api-1        rl-pong-architecture-
metrics-api        "uvicorn src.metrics…"   metrics-api        29
minutes ago   Up 29 minutes             0.0.0.0:8001->8001/tcp,
[::]:8001->8001/tcp
rl-pong-architecture-metrics-consumer-1   rl-pong-architecture-
metrics-consumer   "python -m src.metri…"   metrics-consumer   29
minutes ago   Up 29 minutes
rl-pong-architecture-mlflow-1
ghcr.io/mlflow/mlflow:v2.9.2            "mlflow server --bac…"
mlflow             29 minutes ago   Up 29 minutes
0.0.0.0:5001->5000/tcp, [::]:5001->5000/tcp
rl-pong-architecture-postgres-1           postgres:15-alpine
"docker-entrypoint.s…"   postgres           29 minutes ago   Up 29
minutes (healthy)   5432/tcp
rl-pong-architecture-zookeeper-1          confluentinc/cp-
zookeeper:7.6.1         "/etc/confluent/dock…"   zookeeper
29 minutes ago   Up 29 minutes             2181/tcp, 2888/tcp,
3888/tcp

docker compose run --rm smoke
<…>
[+] run 5/5
SMOKE RESULT: ALL PASS
```





13. ML-часть: датасет, preprocessing, модель

– Датасет: self-generated – траектории взаимодействия агента со средой; постоянного датасета нет по природе RL, буфер реплея 50 000 переходов.

– Preprocessing: grayscale 84×84, стек 4 кадров, значения uint8 0..255.

– Модель: DQN, CnnPolicy SB3 (3 conv-слоя + 2 fc), lr 1e-4, gamma 0.99, buffer 50k, batch 32, exploration 1.0→0.05 за 20% шагов.

– Baseline: случайная политика (равномерные действия).

– Метрики: средняя награда за эпизод (окно 50), количество очков агента, доля выигранных матчей, латентность предикта.

– Результаты demo-прогона (docs/demo_metrics.json): 457 эпизодов, средняя награда первых 50 эпизодов −3.66 → последних 50 +5.66 (improvement +9.32), у random baseline −4.44; в последних 50 эпизодах агент выигрывает 98% матчей.

– Ограничения: обучение CPU-долгое; политика DQN может «застревать» на определённых траекториях подачи; противник-эвристика слабее настоящего игрока – перенос на ALE Breakout требует донастройки.

14. RL-часть: environment, state/action/reward, алгоритм

– Environment: PongEnv (Pygame, Gymnasium API): поле 84×84, две платформы, мяч с ускорением при отбивании; матч до 11 очков. Противник – трекинг с шумом и ограниченной скоростью.

– State: стек 4 кадров 84×84 grayscale (несёт информацию о скорости мяча).

– Action: дискретное {stay, up, down}.

– Reward: +1 забитый гол, −1 пропущенный; shaped-стратегия добавляет +0.01 за выравнивание платформы с мячом (ускоряет раннее обучение).

– Алгоритм: DQN (SB3) с target network и epsilon-greedy; альтернатива – PPO.

– График обучения: docs/training_curve.png (demo-прогон: сглаженная награда растёт с уровня random baseline −4.44 до +5.66, winrate 0.98).

– Реальный DQN-прогон (docker, Kafka → Postgres): последний run c24bbcc8, 52 эпизода: средняя награда по всему прогону −3.17 (старт с нуля), хвост последних 10 эпизодов +9.92; оценка обученной политики (greedy, 5 эпизодов) avg +10.08. Тренировочный mean_reward_last_50 = 8.697. Среднее по всей таблице (−6.355 в /experiments) смешивает 5 прогонов и все этапы обучения и не является итоговой метрикой; ранние прогоны (−9.4…−9.9) – этап нулевой обученности, фактически baseline.

Итоговая сводка метрик:

Показатель	Значение
Random baseline	−4.44
Ранние прогоны DQN (начало обучения)	−9.4…−9.9
Хвост последних 10 эпизодов последнего DQN-прогона	+9.92
Оценка жадной политики (5 эпизодов)	+10.08
Improvement по /summary (last_50 vs first_50)	+15.02

Обучение превышает baseline на ~14 очков награды за эпизод; согласованность трёх независимых источников (SQL-хвост, greedy-оценка, /summary) подтверждает корректность полного pipeline от trainer до аналитического хранилища.

Ограничения RL: sparse reward, длина эпизода, стабилизация таргет-сети; полный DQN-прогон воспроизводится командой docker compose run --rm trainer.

15. Выводы и распределение работы

Выводы:

– Архитектура «тренер → Kafka → хранилище → API» отработала end-to-end: события тренировки и предикты проходят через шину, метрики доступны по API, полный DQN-прогон зафиксирован в PostgreSQL и подтверждён тремя независимыми источниками метрик.

– 5 паттернов применены там, где они убирают реальное дублирование (фабрики, стратегии алгоритмов/награды, телеметрия, состояния матча, команды).

– Обучение даёт измеримый прирост против baseline; оба замечания ревьюера закрыты проверяемыми артефактами, а не декларациями.

Воспроизводимость: demo-цифры получены реальным запуском scripts/demo_tabular.py; полный DQN-прогон выполнен в docker-стеке (метрики в PostgreSQL: run c24bbcc8, см. раздел 14; доказательная база – папка artifacts/). Скриншоты UI снимаются после docker compose up -d --build.